Skill

Topic 1 : To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
vennela_07@VennelaPC:~$ mkdir quote_project
vennela_07@VennelaPC:~$ cd quote_project
vennela_07@VennelaPC:~/quote_project$ nano single_quotes.c
vennela_07@VennelaPC:~/quote_project$ gcc -Wall -Wextra single_quotes.c -o single_quotes
vennela_07@VennelaPC:~/quote_project$ ./single_quotes
Enter a command (type exit to quit):
```

```
> echo 'hello world'

Parsed tokens:
Token 1: [echo]
Token 2: [hello world]
Total tokens: 2
Validation successful.

> echo '$HOME'

Parsed tokens:
Token 1: [echo]
Token 2: [$HOME]
Total tokens: 2
Validation successful.

> echo ''

Parsed tokens:
Token 1: [echo]
Token 2: []
Total tokens: 2
Validation successful.

> echo 'hello
Error: Unclosed single quote.
> 'one two three'

Parsed tokens:
Token 1: [one two three]
Total tokens: 1
Validation successful.

> echo hello' world'
```

```
> echo hello' world'

Parsed tokens:
Token 1: [echo]
Token 2: [hello world]
Total tokens: 2
Validation successful.

> int token_active = 0;

Parsed tokens:
Token 1: [int]
Token 2: [token_active]
Token 3: [=]
Token 4: [0;]
Total tokens: 4
Validation successful.
```

```
  GNU nano 7.2                 single-quotes.c *

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 512
#define MAX_TOKENS 50
#define MAX_TOKEN 200

int main(void) {
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN];

    printf("Enter a command (type exit to quit):\n");
```

```
    while (1) {
        printf("> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        int i = 0, j = 0, count = 0;
        int in_single = 0;
        int token_started = 0;
        int error = 0;

        while (input[i] != '\0') {
            char ch = input[i];

            if (ch == '\'') {
                in_single = !in_single;
                token_started = 1;
                i++;
                continue;
            }

            if (!in_single && (ch == ' ' || ch == '\t')) {
                if (token_started) {
                    tokens[count][j] = '\0';
                    count++;
                    j = 0;
                    token_started = 0;

                    if (count >= MAX_TOKENS) {
                        error = 1;
                        break;
                    }
                }
                i++;
                continue;
                continue;
            }

            if (j >= MAX_TOKEN - 1) {
                error = 1;
                break;
            }

            tokens[count][j++] = ch;
            token_started = 1;
            i++;
        }

        if (in_single) {
            printf("Error: Unclosed single quote.\n");
            continue;
        }

        if (error) {
            printf("Error: Input or token limit exceeded.\n");
            continue;
        }
```

```
        }

        if (token_started) {
            tokens[count][j] = '\0';
            count++;
        }

        printf("\nParsed tokens:\n");

        for (int k = 0; k < count; k++) {
            printf("Token %d: [%s]\n", k + 1, tokens[k]);
        }

        printf("Total tokens: %d\n", count);
        printf("Validation successful.\n\n");
    }

    printf("Parser terminated.\n");
    return 0;
}
```

Topic 2 : To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
vennela_07@VennelaPC:~$ mkdir double_quote_project
vennela_07@VennelaPC:~$ cd double_quote_project
vennela_07@VennelaPC:~/double_quote_project$ nano double_quotes.c
vennela_07@VennelaPC:~/double_quote_project$ gcc -Wall -Wextra double_quotes.c -o double_quotes
vennela_07@VennelaPC:~/double_quote_project$ ./double_quotes
Double-Quote Parser
Enter a command (type exit to quit).
> echo "hello world"

Parsed tokens:
Token 1: [echo]
Token 2: [hello world]
Total tokens: 2
Validation successful.

> echo "$HOME"

Parsed tokens:
Token 1: [echo]
Token 2: [/home/vennela_07]
Total tokens: 2
Validation successful.

> Token 1: [echo]

Parsed tokens:
Token 1: [Token]
Token 2: [1:]
Token 3: [[echo]]
Total tokens: 3
Validation successful.

> Token 2: [/home/student]
```

```
Token 2: [2:]
Token 3: [[/home/student]]
Total tokens: 3
Validation successful.

> echo "${USER}"

Parsed tokens:
Token 1: [echo]
Token 2: [vennela_07]
Total tokens: 2
Validation successful.

> echo hello" world"

Parsed tokens:
Token 1: [echo]
Token 2: [hello world]
Total tokens: 2
Validation successful.

> Token 1: [echo]

Parsed tokens:
Token 1: [Token]
Token 2: [1:]
Token 3: [[echo]]
Total tokens: 3
Validation successful.

> Token 2: [hello world]

Parsed tokens:
Token 1: [Token]
Token 2: [2:]
Token 3: [[hello]
Token 4: [world]]
Total tokens: 4
Validation successful.

> echo "hello world
Error: Invalid or unclosed quote/variable.
> exit
Parser terminated.
```

Topic 3 : To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
vennela_07@VennelaPC:~$ mkdir escape_sequence_project
cd escape_sequence_project
vennela_07@VennelaPC:~/escape_sequence_project$ nano escape_sequences.c
vennela_07@VennelaPC:~/escape_sequence_project$ gcc -Wall -Wextra escape_sequences.c -o escape_sequences
vennela_07@VennelaPC:~/escape_sequence_project$ ./escape_sequences
Escape Sequence Parser
Use backslash to escape characters.
Type exit to quit.
```

```
> file\&name test\;value

Parsed tokens:
Token 1: [file&name]
Token 2: [test;value]
Total tokens: 2
Validation successful.

> file\&name test\;value

Parsed tokens:
Token 1: [file&name]
Token 2: [test;value]
Total tokens: 2
Validation successful.

> hello\\world

Parsed tokens:
Token 1: [hello\world]
Total tokens: 1
Validation successful.

> my\ file\ name\ \&\ test

Parsed tokens:
Token 1: [my file name & test]
Total tokens: 1
Validation successful.

> hello\
Error: Incomplete escape sequence.
> exit
Parser terminated.
```

```
 GNU nano 7.2                                                escape_sequences.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define INPUT_SIZE 512
#define MAX_TOKENS 50
#define TOKEN_SIZE 256

int main(void) {
    char input[INPUT_SIZE];

    printf("Escape Sequence Parser\n");
    printf("Use backslash to escape characters.\n");
    printf("Type exit to quit.\n");

    while (1) {
        printf("> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
```

```
        if (strchr(input, '\n') == NULL && !feof(stdin)) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {}
            printf("Error: Input too long.\n");
            continue;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        char tokens[MAX_TOKENS][TOKEN_SIZE] = {{0}};
        int count = 0;
        int active = 0;
        int error = 0;
        size_t len = 0;

        for (size_t i = 0; input[i] != '\0'; i++) {
            char ch = input[i];

            if (ch == '\\') {
                if (input[i + 1] == '\0') {
                    error = 1;
                    break;
                }

                char next = input[++i];

                if (count >= MAX_TOKENS ||
                    len >= TOKEN_SIZE - 1) {
                    error = 2;
                    break;
                }

                tokens[count][len++] = next;
                tokens[count][len] = '\0';
                active = 1;
                continue;
            }

            if (isspace((unsigned char)ch)) {
                if (active) {
                    if (count >= MAX_TOKENS) {
                        error = 2;
                        break;
                    }

                    tokens[count++][len] = '\0';
                    len = 0;
                    active = 0;
                }
                continue;
            }

            if (count >= MAX_TOKENS ||
                len >= TOKEN_SIZE - 1) {
                error = 2;
                break;
            }

            tokens[count][len++] = ch;
            tokens[count][len] = '\0';
            active = 1;
        }
```

```c
        if (error) {
            if (error == 1)
                printf("Error: Incomplete escape sequence.\n");
            else
                printf("Error: Token or token-size limit exceeded.\n");
            continue;
        }

        if (active) {
            if (count >= MAX_TOKENS) {
                printf("Error: Too many tokens.\n");
                continue;
            }
            tokens[count++][len] = '\0';
        }

        printf("\nParsed tokens:\n");

        for (int i = 0; i < count; i++) {
            printf("Token %d: [%s]\n", i + 1, tokens[i]);
        }

        printf("Total tokens: %d\n", count);
        printf("Validation successful.\n\n");
    }

    printf("Parser terminated.\n");
    return 0;
}
```

Topic 4 : To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
vennela_07@VennelaPC:~$ mkdir child_process_project
cd child_process_project
vennela_07@VennelaPC:~/child_process_project$ nano child_process.c
vennela_07@VennelaPC:~/child_process_project$ gcc -Wall -Wextra child_process.c -o child_process
vennela_07@VennelaPC:~/child_process_project$ ./child_process ls
Parent process started.
Parent PID: 1417
Parent is waiting for child PID 1418.
Child process created.
Child PID: 1418
Executing command: ls
child_process  child_process.c
Child exited with status: 0
Parent process completed.
vennela_07@VennelaPC:~/child_process_project$ ./child_process ls -l
Parent process started.
Parent PID: 1419
Parent is waiting for child PID 1420.
Child process created.
Child PID: 1420
Executing command: ls
total 24
-rwxr-xr-x 1 vennela_07 vennela_07 16408 Oct 13 05:48 child_process
-rw-r--r-- 1 vennela_07 vennela_07  1430 Oct 13 05:48 child_process.c
Child exited with status: 0
Parent process completed.
```

```
vennela_07@VennelaPC:~/child_process_project$ ./child_process ls -l
Parent process started.
Parent PID: 1421
Parent is waiting for child PID 1422.
Child process created.
Child PID: 1422
Executing command: ls
total 24
```

```
-rwxr-xr-x 1 vennela_07 vennela_07 16408 Oct 13 05:48 child_process
-rw-r--r-- 1 vennela_07 vennela_07  1430 Oct 13 05:48 child_process.c
Child exited with status: 0
Parent process completed.
vennela_07@VennelaPC:~/child_process_project$ ./child_process invalidcommand
Parent process started.
Parent PID: 1424
Parent is waiting for child PID 1425.
Child process created.
Child PID: 1425
Executing command: invalidcommand
execvp failed: No such file or directory
Child exited with status: 127
Parent process completed.
vennela_07@VennelaPC:~/child_process_project$ ./child_process
Usage: ./child_process command [arguments...]
Example: ./child_process ls -l
```

```
  GNU nano 7.2                                                    child_process.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    pid_t pid;
    int status;

    if (argc < 2) {
        printf("Usage: %s command [arguments...]\n", argv[0]);
        printf("Example: %s ls -l\n", argv[0]);
        return 1;
    }

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", (int)getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("Child process created.\n");
        printf("Child PID: %d\n", (int)getpid());
        printf("Executing command: %s\n", argv[1]);
        fflush(stdout);

        execvp(argv[1], &argv[1]);
```

```c
        /* Runs only if execvp fails */
        perror("execvp failed");
        _exit(127);
    } else {
        /* Parent process */
        printf("Parent is waiting for child PID %d.\n", (int)pid);

        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid failed");
            return 1;
        }

        if (WIFEXITED(status)) {
            printf("Child exited with status: %d\n",
                   WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Child terminated by signal: %d\n",
                   WTERMSIG(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
```